

DESENVOLVIMENTO MOBILE

AULA 3 – CORE COMPONENTS, STYLESHEET E FLEXBOX

CESAR School · 2026.2



NA AULA 1 VOCÊS DESCOBRIRAM QUE `<View>` NÃO É `<div>`.

HOJE VEM A CONSEQUÊNCIA: TODO O VOCABULÁRIO MUDOU — E A CASCATA NÃO EXISTE.

AGENDA – 3 HORAS

Bloco	Tempo
Abertura + o ambiente corrigido	10 min
1. O catálogo mínimo: Core Components	35 min
☕ Intervalo	10 min
2. StyleSheet: o estilo que não é CSS	30 min
3. Checkpoint conceitual	10 min
4. Flexbox: como o espaço é negociado	35 min
5. Onde os estilos devem morar	20 min
6. Hands-on guiado	25 min
Fechamento	5 min



AO FINAL DA AULA VOCÊ DEVE CONSEGUIR

- 1 **Escolher** o componente certo — e justificar a escolha
- 2 **Aplicar** estilo com `StyleSheet` : condicional e precedência
- 3 **Explicar** por que não há cascata, e qual é a única exceção
- 4 **Ligar estado a interface** com `useState` , `onPress` e componentes controlados
- 5 **Construir** um layout Flexbox **prevendo** o resultado antes de rodar
- 6 **Decidir** onde cada estilo do projeto deve morar
- 7 **Corrigir** o que a maioria dos tutoriais ainda ensina errado

PARTE 0

0 AMBIENTE, ATUALIZADO

CRIAR E LIMPAR UM PROJETO

```
# SDK atual: 57 = React Native 0.86.2 + React 19.2
npx create-expo-app@latest meu-app

cd meu-app
npm run reset-project # move o exemplo p/ app-example/, deixa app/ limpa
npx expo start
```

Depois do reset você pode **apagar** `app-example/`.

Templates: `default` (recomendado) · `blank` · `blank-typescript` · `tabs` · `bare-minimum`

EXPO GO NO IPHONE: A REGRA DE VERDADE

Onde rodar	O que fazer
Android (aparelho/emulador)	<code>expo.dev/go</code> — build do seu SDK
Simulador iOS	<code>expo.dev/go</code>
iPhone físico, SDK 55+	<code>sign.expo.dev</code> — Apple ID grátis, cert ~7 dias
iPhone físico, SDK 54	Expo Go da App Store
iPhone físico, SDK ≤ 53	Não dá — atualize ou faça dev build

A APP STORE ESTÁ NO 54 PORQUE A BUILD DE 57 **AGUARDA APROVAÇÃO.**



UMA CORREÇÃO DE COMANDO

```
# ❌ O que o material antigo mandava  
npm install react-dom@19.2.0 react-native-web@^0.21.0  
  
# ✅ O certo  
npx expo install react-dom react-native-web @expo/metro-runtime  
npx expo start --web
```

npx expo install RESOLVE A VERSÃO **COMPATÍVEL COM O SEU SDK**.

npm install pega a mais recente do npm — que pode ser de um SDK futuro.

PARTE 1

CORE COMPONENTS

O DICIONÁRIO DE TRADUÇÃO

Web	React Native	Lembrar
<code><div></code>	View	não tem scroll implícito
<code><p></code> <code></code> <code><h1></code>	Text	todo texto vive aqui
<code></code>	Image	remota precisa de dimensão
<code><input type="text"></code>	TextInput	<code>value</code> + <code>onChangeText</code>
<code><input type="checkbox"></code>	Switch	<code>value</code> + <code>onValueChange</code>
scroll da página	ScrollView	renderiza tudo ; duas props de estilo
<code><button></code>	Button	do sistema, não aceita <code>style</code>
<code>className</code> / <code>.css</code>	StyleSheet	objetos JS

SÃO **SETE**. O DICIONÁRIO DA AULA CABE NUMA MÃO E MEIA.





O DICIONÁRIO DE VIAGEM

VOCÊ JÁ FALA A LÍNGUA. RECEBEU UM DICIONÁRIO MENOR, COM PALAVRAS DIFERENTES PARA AS MESMAS IDEIAS.

E algumas palavras não têm tradução: não existe `<h1>` – existe `<Text>` com um estilo.

Text – REGRA 1

```
<View>Beber água</View>           // ✗ quebra em runtime  
<View><Text>Beber água</Text></View> // ✓
```

Text strings must be rendered within a <Text> component

VOCÊ VAI VER ESSE ERRO. TODO MUNDO VÊ.

Text – REGRA 2: A ÚNICA HERANÇA QUE EXISTE

```
// A View NÃO passa fontSize para o Text
<View style={{ fontSize: 24 }}>           // ✗ não faz nada
  <Text>Continuo no tamanho default</Text>
</View>

// Text dentro de Text, sim
<Text style={{ fontSize: 24 }}>
  Grande <Text style={{ fontWeight: 'bold' }}>e em negrito</Text>
</Text>
```

<Text> DENTRO DE **<Text>** É A ÚNICA HERANÇA DE ESTILO EM REACT NATIVE.

Image – A QUE PRECISA DE DIMENSÃO

```
// Local: o bundler já sabe o tamanho em tempo de build
<Image source={require('../assets/habito.png')} />

// Remota: VOCÊ precisa informar
<Image
  source={{ uri: 'https://exemplo.com/habito.png' }}
  style={{ width: 64, height: 64, borderRadius: 32 }}
/>
```

SEM `width` / `height`, A IMAGEM REMOTA TEM TAMANHO **ZERO**.

Nada aparece. E **não há erro no console**.

Image – EXISTE ALGO MELHOR NO EXPO

```
npx expo install expo-image
```

- Cache em **disco e memória**
- **Transição** suave na troca de imagem – sem flicker
- Placeholders **BlurHash / ThumbHash**
- WebP, AVIF, HEIC, SVG, GIF animado

A doc do React Native **não** deprecia o `Image` nativo – mas para imagem de rede em produção, `expo-image` é o caminho.

TextInput – A ENTRADA CONTROLADA

```
const [titulo, setTitulo] = useState('');

<TextInput
  style={styles.input}           // sem estilo, ele é INVISÍVEL
  value={titulo}
  onChangeText={setTitulo}       // recebe a string, não um evento
  placeholder="Ex.: Beber 2L de água"
  maxLength={60}
/>
```

- É **onChangeText** – sem **e.target.value**
- **Não tem borda por default.** Se você não estilizar, não vê nada
- Úteis: **keyboardType** · **secureTextEntry** · **multiline**

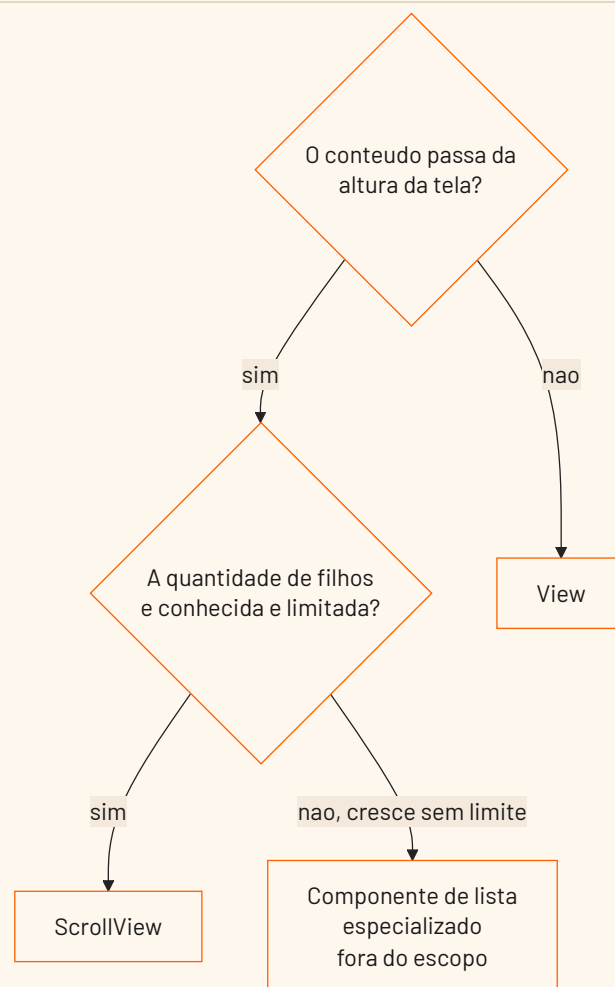
ScrollView – A PEGADINHA DAS DUAS PROPS DE ESTILO

```
<ScrollView
  style={styles.tela}           // a JANELA de rolagem
  contentContainerStyle={styles.conteudo} // o CONTEÚDO que rola
>
```

Quer isso	Vai em
padding no conteúdo	contentContainerStyle
centralizar os filhos	contentContainerStyle
altura / fundo da janela	style

alignItems NO style DE UM ScrollView NÃO FAZ NADA.

View OU ScrollView ?



`ScrollView` monta **todos** os filhos de uma vez. Com 20, ok. Com 2.000, o app engasga.

Caso típico de `ScrollView`: um **formulário** — campos conhecidos, quantidade fixa.

Button – O BOTÃO DO SISTEMA

```
<Button title="Salvar" onPress={salvar} color="#FF6002" disabled={false} />
```

São **essas** as props: `title` · `onPress` · `color` · `disabled`

Plataforma	O que color tinge
iOS	o texto do botão
Android	o fundo do botão

O MESMO `color` PRODUZ DOIS BOTÕES DIFERENTES. **TESTE NAS DUAS.**





Button NÃO ACEITA **style**.

ELE ENTREGA O BOTÃO DO SISTEMA. VOCÊ ESCOLHE TEXTO, AÇÃO E UMA COR.



O QUE DÁ PARA FAZER HOJE: ESTILIZAR EM VOLTA

```
<View style={styles.areaBotao}>
  <Button title="Marcar concluído" onPress={concluir} color={cores.primaria} />
</View>

// ...
areaBotao: {
  borderRadius: 8,
  overflow: 'hidden',      // no Android, recorta o fundo no raio
  marginTop: espaco.sm,    // margin EM VOLTA do grupo: uso legítimo
},
```

VOCÊ NÃO CONTROLA O BOTÃO. **CONTROLA O ESPAÇO DELE.**

Text COM onPress – TOQUE DENTRO DO ESCOPO

```
const [concluido, setConcluido] = useState(false);

<Text
  style={[styles.titulo, concluido && styles.tituloConcluido]}
  onPress={() => setConcluido(anterior => !anterior)}
>
  Beber 2L de água
</Text>
```

TOQUE MUDA O ESTADO → ESTADO MUDA O ARRAY DE ESTILOS → A TELA REFLETE.

Você já conhece as duas primeiras partes. A terceira é a novidade de hoje.

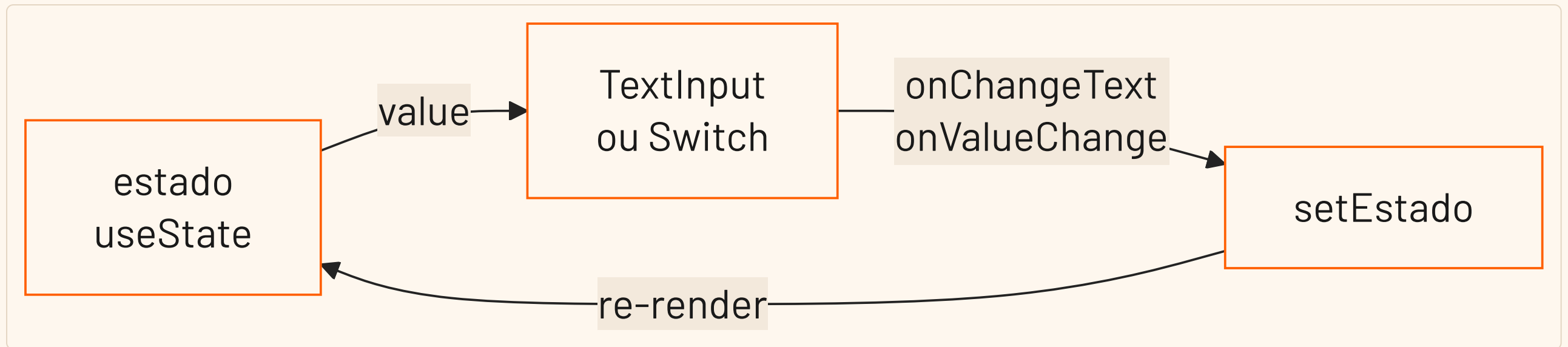
Switch – O LIGA/DESLIGA

```
const [lembrete, setLembrete] = useState(false);

<View style={styles.linha}>
  <Text>Lembrete diário</Text>
  <Switch
    value={lembrete}
    onChange={setLembrete} // entrega o booleano novo direto
    trackColor={{ false: '#ccc', true: '#FFB132' }}
  />
</View>
```

`onChange={setLembrete}` funciona sem função intermediária – mesmo padrão do `onChangeText`.

O LAÇO DO COMPONENTE CONTROLADO



O componente **não guarda** o valor. O estado é a **fonte de verdade**.

ESQUECEU O value ? O USUÁRIO MEXE E A TELA VOLTA AO VALOR ANTERIOR.



INTERVALO

10 MINUTOS

PARTE 2

STYLESHEET



NÃO EXISTE CASCATA.

NÃO EXISTE HERANÇA DE ESTILO.

(exceto `Text` dentro de `Text`)



A ANALOGIA DO CÓDIGO DE VESTIMENTA

NO CSS

Você anuncia a regra para o prédio inteiro:

"todo texto deste site é Arial, 16px"

Todo mundo lá dentro **herda** o anúncio.

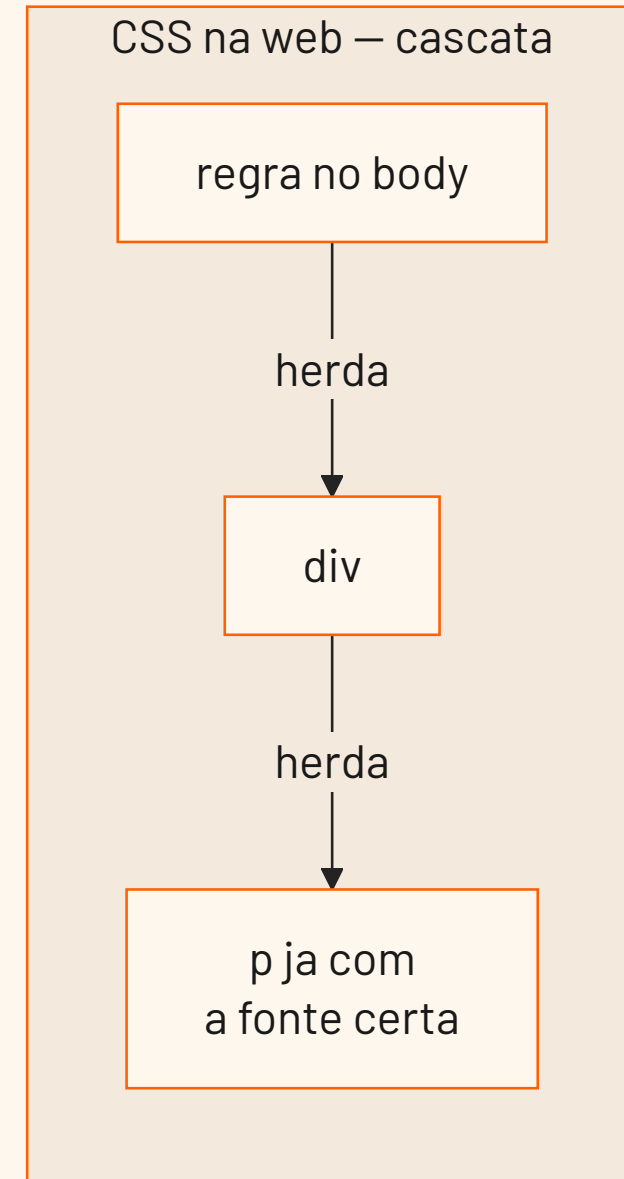
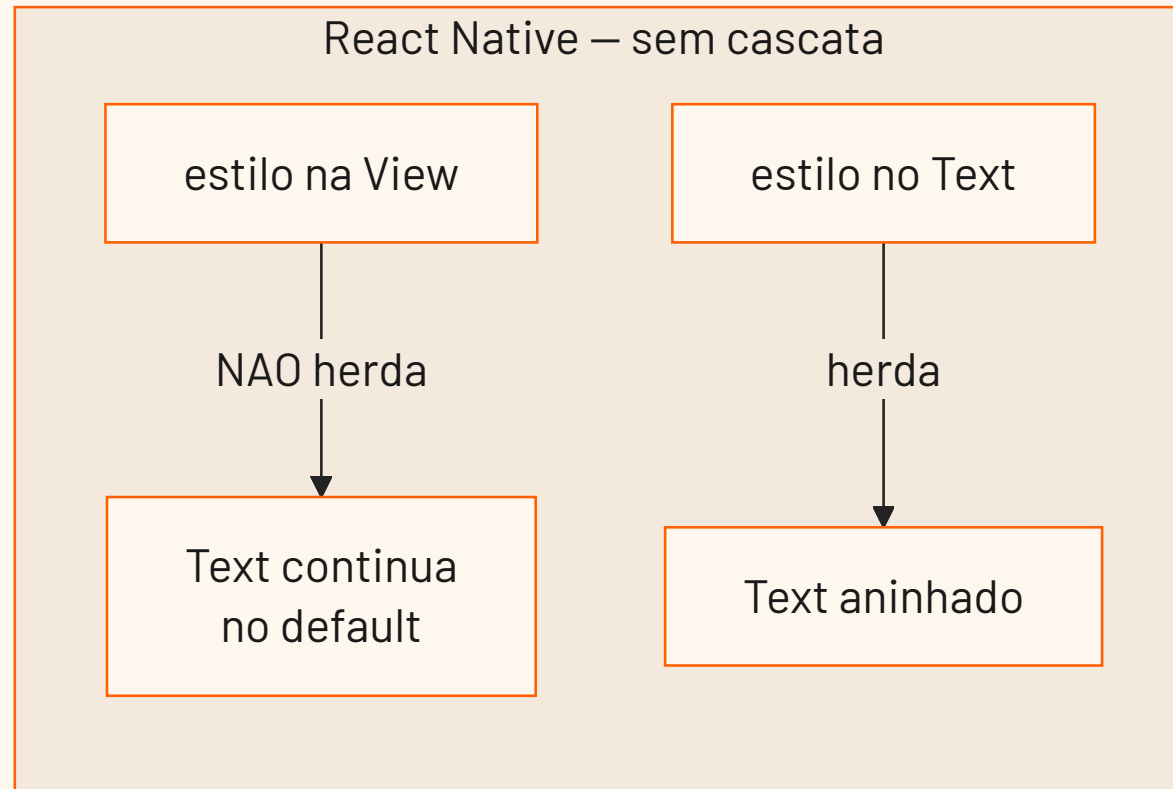
EM REACT NATIVE

Não existe alto-falante.

Cada componente **se veste sozinho**.

Cinco textos iguais = cinco vezes o estilo.

CASCATA VS. SEM CASCATA



No bloco do CSS o estilo desce sozinho pela árvore. No bloco do React Native, a única seta de herança vai de **Text** para **Text**.

AS OUTRAS DIFERENÇAS DE FUNDO

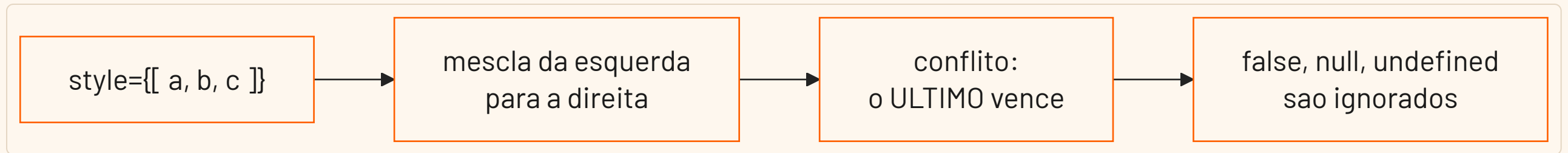
```
{ padding: 16 }           // ✓ número puro – são dp
{ padding: '16px' }       // ✗ unidade CSS não existe
{ width: '100%' }         // ✓ percentual funciona, como string
{ fontSize: '1.5rem' }    // ✗ não há rem, em, vh, vw
```

- **Sem seletores** – nada de `.classe`, `#id`, `>`, `:first-child`
- **Sem `:hover`** – não há mouse. Há `pressed`
- **`camelCase`** – `backgroundColor`, não `background-color`
- **Subconjunto** do CSS

AS FORMAS DE PASSAR **style**

```
<View style={styles.card} /> // objeto
<View style={[styles.card, styles.destacado]} /> // array
<Text style={[styles.texto, ativo && styles.ativo]} /> // condicional
<View style={[styles.badge, { backgroundColor: cor }]} /> // runtime
<View style={[styles.card, // vários
  concluido && styles.cardConcluido,
  urgente && styles.cardUrgente]} />
```

PRECEDÊNCIA: O ÚLTIMO VENCE



```
<Text style={{ { color: 'blue' }, { color: 'red' } }}>Que cor?</Text>
```

VERMELHO. É `Object.assign({}, a, b)`.

O QUE `StyleSheet.create` FAZ

✓ **Valida em tempo de tipo** e dá autocomplete
→ `backgroundColour` vira erro de compilação

✗ **Não faz mágica de performance**

A história antiga: ele registrava os estilos e passava só um **ID pela ponte**. A ponte **não existe mais** — foi removida na 0.84 (Aula 1, §3.4).

HOJE O GANHO É VALIDAÇÃO E ORGANIZAÇÃO.

FATO ZUMBI: O `as const`

```
// ❌ O que o material antigo escrevia
StyleSheet.create({
  container: { justifyContent: 'center' as const },
});

// ✅ O que basta
StyleSheet.create({
  container: { justifyContent: 'center' },
});
```

A assinatura de `create` **já preserva os literais**.

Onde ele importa: objeto solto, **fora** de `create` — aí o TS alarga `'center'` para `string`.

APRENDA A DISTINÇÃO, NÃO A RECEITA.



FATO ZUMBI: A SOMBRA

```
// ❌ Cinco props, resultado diferente em cada plataforma
{ shadowColor: '#000', shadowOffset: { width: 0, height: 2 },
  shadowOpacity: 0.1, shadowRadius: 4, elevation: 5 }

// ✅ Uma linha, as duas plataformas
{ boxShadow: '0px 2px 8px rgba(0, 0, 0, 0.12)' }
```

`boxShadow` funciona na **New Architecture** — o **único modo** desde a **0.82**, e desde a **0.84** a legada nem existe mais no código. No projeto de vocês, funciona.

Também chegou `filter`: blur, brightness, saturate.

PARTE 3

CHECKPOINT

RESPONDA EM DUPLA – 60 SEGUNDOS CADA

- 1 Coloquei `fontSize: 24` na `View` e o texto não mudou. **Por quê?**
- 2 Onde vai o `padding` de um `ScrollView`? **Por que não no `style`?**
- 3 Quero um botão com fundo gradiente e canto arredondado. `<Button>`?
- 4 Em `style={ [a, b] }`: `a` diz `color: 'blue'`, `b` diz `color: 'red'`. **Qual vence?** E se `b` for `false`?

PARTE 4

FLEXBOX



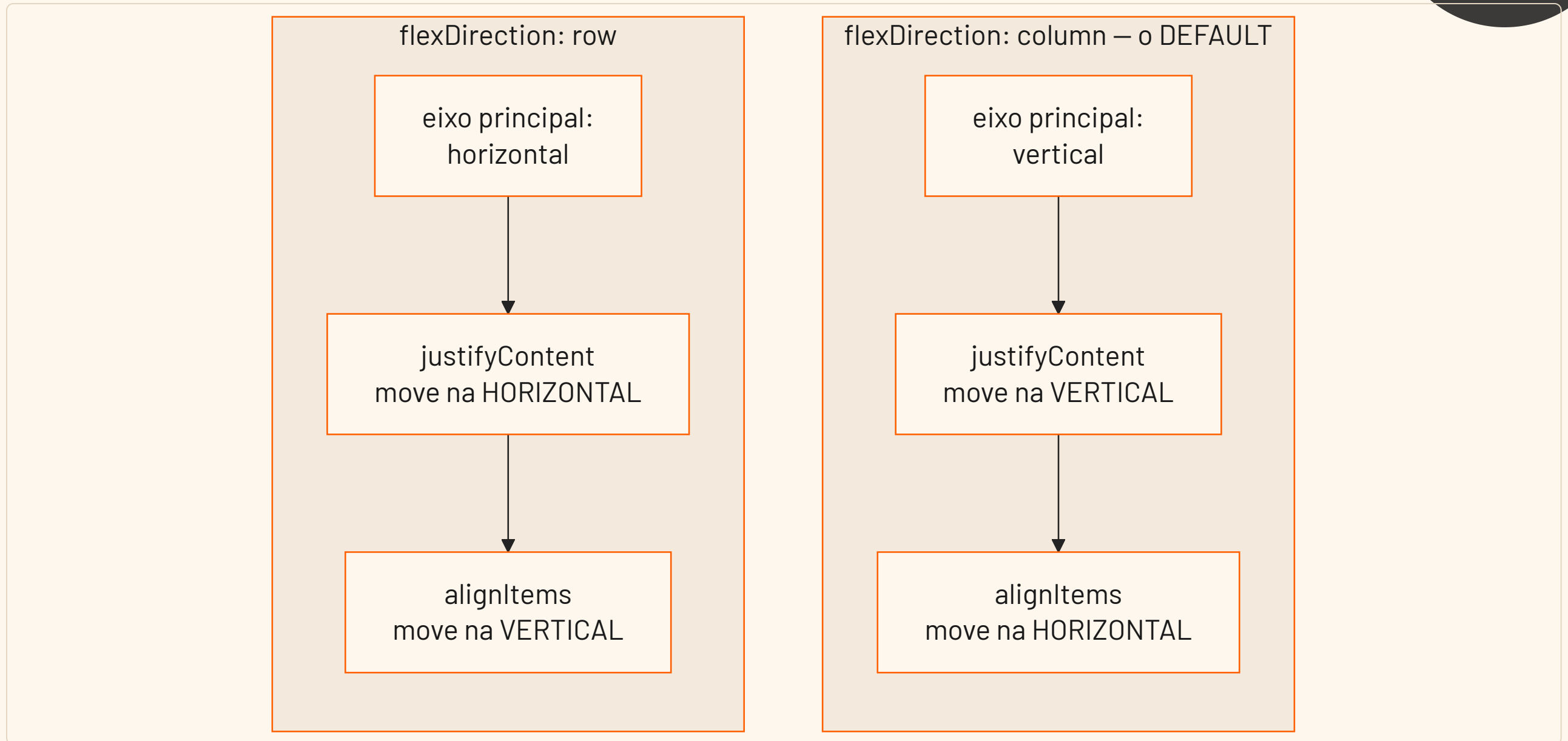
A ANALOGIA DA FILA

- **flexDirection** — em que direção a fila anda?
- **justifyContent** — como as pessoas se distribuem **ao longo** da fila?
- **alignItems** — onde elas ficam **na largura** da fila?

TODA DÚVIDA DE FLEXBOX SE RESOLVE COM UMA PERGUNTA:

"QUAL É O EIXO PRINCIPAL AQUI?"

OS EIXOS TROCAM DE LUGAR



NÃO EXISTE "justifyContent É HORIZONTAL". DEPENDE DA DIREÇÃO.



flexDirection DEFAULT É **COLUMN**.

NA WEB É **row**.

Este é o bug de layout nº 1 do semestre.



O QUE ACONTECE NA PRÁTICA

```
// Você quer categoria e status lado a lado
linha: {
  justifyContent: 'space-between', // ❌ distribui na VERTICAL
}

// Corrigido
linha: {
  flexDirection: 'row', // ✅
  justifyContent: 'space-between',
}
```

SE ALGO "NÃO VAI PARA O LADO", CHECAR **flexDirection** PRIMEIRO.

Segundo default que surpreende: `alignItems` é **stretch** — filho sem largura ocupa a largura toda do pai.

justifyContent – NO EIXO PRINCIPAL

Valor	Efeito
flex-start (default)	tudo no começo
center	tudo no meio
flex-end	tudo no fim
space-between	pontas nas bordas, sobra dividida no meio
space-around	espaço em volta de cada item – bordas com metade
space-evenly	vãos exatamente iguais, bordas incluídas

alignItems – no eixo cruzado: stretch (default) · flex-start · center · flex-end · baseline

CENTRALIZAR NO MEIO DA TELA

```
tela: {  
  flex: 1,           // ← SEM isto, nada acontece  
  justifyContent: 'center', // centro vertical (eixo principal)  
  alignItems: 'center',   // centro horizontal (eixo cruzado)  
}
```

SEM `flex: 1`, A `View` TEM SÓ A ALTURA DO CONTEÚDO.

Centralizar dentro de uma caixa do tamanho exato do conteúdo não muda nada — **não há sobra para distribuir.**

flex, flexGrow, flexBasis

```
{ flex: 1 }           // "ocupe o que sobrar"
                      // = flexGrow:1 + flexShrink:1 + flexBasis:0

{ flexGrow: 2 }       // proporção da SOBRA
{ flexGrow: 1 }       // → 2:1:1
{ flexGrow: 1 }

{ flexBasis: 100 } // tamanho INICIAL no eixo principal
```

- **flexBasis** = ponto de partida
- **flexGrow** = como a sobra é dividida

📌 Em RN, flexShrink default é **0** (na web é **1**). Por default os itens **não** encolhem.

GRADE: flexWrap + gap

```
grade: {
  flexDirection: 'row',
  flexWrap: 'wrap',
  gap: 12,
},
celula: {
  flexBasis: '30%',
  flexGrow: 1,           // absorve a sobra da linha
  aspectRatio: 1,       // quadrado sem calcular altura
}
```

`aspectRatio` poupa a conta de altura. `flexGrow` faz as células fecharem a largura **sem percentual na mão** — sobrevive a mudança de tamanho de tela.

FATO ZUMBI: ESPAÇAR COM `margin`

```
// ❌ margem em cada filho  
celula: { margin: 5 }
```

Entre dois vizinhos as margens **somam**: $5 + 5 = 10\text{px}$.
Contra a borda há só **uma**: **5px**.
→ o vão interno é o **dobro** do externo. Layout irregular.

```
// ✅ gap no contêiner: espaço só ENTRE os filhos  
grade: { gap: 10 }
```

`gap` · `padding` · `margin` = ENTRE IRMÃOS · DENTRO DO CONTÊINER · EM VOLTA DO GRUPO

FLEXBOX: REACT NATIVE VS CSS

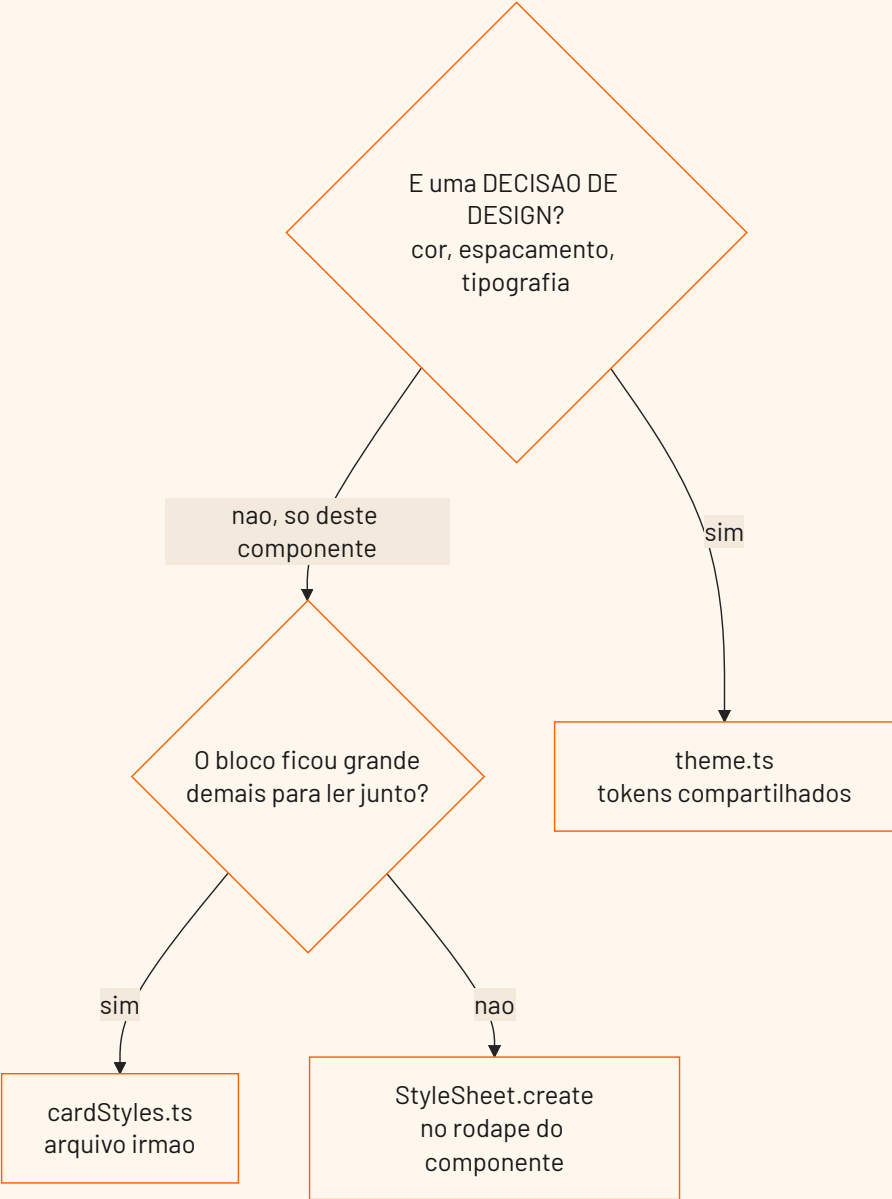
	React Native	CSS
<code>flexDirection</code> default	column	<code>row</code>
<code>flexShrink</code> default	0	<code>1</code>
<code>alignContent</code> default	<code>flex-start</code>	<code>stretch</code>
Unidades	número (dp) ou <code>'%'</code>	<code>px</code> , <code>rem</code> , <code>vh</code> ...
<code>display: grid</code>	não existe	existe
<code>float</code>	não existe	existe
<code>position</code>	<code>relative</code> <code>absolute</code> <code>static</code>	+ <code>fixed</code> <code>sticky</code>

 **Para casa:** [Flexbox Froggy](#) (~20 min). Mas ele é **CSS** — lá o default é `row` .

PARTE 5

ONDE OS ESTILOS DEVEM MORAR

A DECISÃO, EM DUAS PERGUNTAS



ESTILO ESPECÍFICO FICA JUNTO DO COMPONENTE. **DECISÃO DE DESIGN VIRA TOKEN.**

O DEFAULT: NO RODAPÉ DO COMPONENTE

```
export function CardHbito({ titulo, categoria }: CardHbitoProps) {  
  return (  
    <View style={styles.card}>  
      <Text style={styles.titulo}>{titulo}</Text>  
      <Text style={styles.meta}>{categoria}</Text>  
    </View>  
  );  
}  
  
const styles = StyleSheet.create({  
  card: { padding: 16, borderRadius: 12, backgroundColor: '#fff' },  
  titulo: { fontSize: 18, fontWeight: '600' },  
  meta: { fontSize: 12, color: '#666' },  
});
```

É o padrão da comunidade e da doc oficial. **Comece sempre por aqui.**

theme.ts – TOKENS

```
export const cores = {  
  fundo: '#FEF7EE', cartao: '#FFFFFF', texto: '#191919',  
  textoFraco: '#6b6459', primaria: '#FF6002', sucesso: '#2e7d32',  
} as const;  
  
export const espaco = { xs: 4, sm: 8, md: 16, lg: 24, xl: 32 } as const;  
  
export const tipografia = {  
  titulo: { fontSize: 20, fontWeight: '600' },  
  corpo: { fontSize: 16 },  
  legenda: { fontSize: 12, color: cores.textoFraco },  
} as const;
```

 **Aqui o as const faz sentido** — é objeto solto, fora de create . Sem ele, '600' alarga para string , que não é um fontWeight válido.

TOKENS SÃO VALORES.

NÃO SÃO LAYOUTS PRONTOS.

cores.primaria ✓ · espaco.md ✓
globalStyles.container ✗

O ANTI-PADRÃO QUE QUASE TODO MUNDO COMETE

```
// ❌ Isso não é design system – é folha de estilo global com outro nome
export default StyleSheet.create({
  container: { flex: 1, padding: 20 },
  text: { fontSize: 18, color: '#333' },
  button: { backgroundColor: 'blue', padding: 10, borderRadius: 5 },
});
```

Por que dói: `container` é **layout**, não decisão de design. Quando uma tela precisar de `padding: 8`, você cria `container2` – ou sobrescreve com `[globalStyles.container, {padding: 8}]` e perde a rastreabilidade.

VOCÊ ACOPLOU TELAS QUE NÃO TÊM NADA A VER UMA COM A OUTRA.

Quer compartilhar um card? Compartilhe um **componente** `<Card>`, não um objeto de estilo.

OBJETO INLINE: QUANDO É LEGÍTIMO

```
// ✅ 0 valor só existe em runtime
<View style={[styles.badge, { backgroundColor: corDoStatus(status) }]} />

// ❌ Valores fixos soltos no JSX
<View style={{ padding: 16, borderRadius: 12, backgroundColor: '#fff' }} />
```

O objeto é recriado a cada render. Num card isolado ninguém nota; numa tela com centenas de elementos, pesa.

Mas o problema principal **não é performance** — é o número mágico espalhado pelo JSX, longe de qualquer decisão de design.

E O TAILWIND? (CONTEXTO, FORA DO ESCOPO)

Opção	Situação em ago/2026
NativeWind v4	Estável — mas exige Tailwind v3
NativeWind v5	Preview, mira Tailwind v4 — não estável
Uniwind	Novo, para Tailwind v4, API <code>className</code> compatível

A DISCIPLINA USA `StyleSheet` .

É o que a doc oficial ensina, é o que existe em qualquer código base, e é **a base sobre a qual essas bibliotecas são construídas.**

JUNTANDO TUDO

O CARD ESTILIZADO

O CARD ESTILIZADO – O ESTILO

```
const styles = StyleSheet.create({
  tela: { flex: 1, backgroundColor: cores.fundo },
  conteudo: {                                     // ← ScrollView: padding AQUI
    padding: espaco.md, paddingTop: 48,
    gap: espaco.lg,                               // ← gap, não margin
  },
  card: {
    padding: espaco.md, borderRadius: 12,
    backgroundColor: cores.cartao,
    boxShadow: '0px 2px 8px rgba(0,0,0,0.10)',    // ← uma linha
    gap: espaco.sm,
  },
  cardConcluido: { backgroundColor: '#f2f8f2' },
  tituloConcluido: { textDecorationLine: 'line-through' },
  linha: {
    flexDirection: 'row',                        // ← explícito
    justifyContent: 'space-between',
    alignItems: 'center',
  },
  areaBotao: { borderRadius: 8, overflow: 'hidden' },
});
```

0 CARD ESTILIZADO – 0 COMPONENTE

```
export default function TelaHabitDoDia() { // ← sem JSX.Element
  const [status, setStatus] = useState<StatusHabit>(MOCK.status);
  const [lembrete, setLembrete] = useState(false);

  const concluido = status === 'concluido';
  const alternar = () => setStatus(concluido ? 'pendente' : 'concluido');

  return (
    <ScrollView style={styles.tela} contentContainerStyle={styles.conteudo}>
      <View style={[styles.card, concluido && styles.cardConcluido]}>
        <Text
          style={[styles.titulo, concluido && styles.tituloConcluido]}
          onPress={alternar}
        >
          {MOCK.titulo}
        </Text>
        <View style={styles.linha}>
          <Text style={styles.meta}>Lembrete diário</Text>
          <Switch value={lembrete} onValueChange={setLembrete} />
        </View>
        <View style={styles.areaBotao}>
          <Button title={concluido ? 'Desmarcar' : 'Marcar concluído'} onPress={alternar} />
        </View>
      </View>
    </ScrollView>
  );
}
```

SETE COISAS NESSE CÓDIGO

- 1 `gap` no card e no conteúdo — nenhuma `margin` para espaçar irmãos
- 2 `boxShadow` em uma linha, no lugar do quarteto + `elevation`
- 3 `flexDirection: 'row'` explícito na linha
- 4 O `padding` foi para `contentContainerStyle`, não para o `style`
- 5 Um estado, dois efeitos visuais — `concluido` aparece em **dois** arrays
- 6 `Text` com `onPress` e `Button` embrulhado numa `View` estilizada
- 7 Nenhum `as const` dentro de `create` e nenhum `: JSX.Element`

OS FATOS ZUMBIS DESTA AULA

O material antigo ensina	O que vale hoje
<code>JSX.Element</code> como tipo de retorno	React 19 removeu o <code>JSX</code> global — omita a anotação
as <code>const</code> dentro de <code>create</code>	Desnecessário; <code>create</code> preserva literais
<code>--template default@sdk-54</code> fixo	SDK 57 ; iPhone com SDK 55+ via <code>sign.expo.dev</code>
<code>npm install react-dom@19.2.0</code>	<code>npx expo install</code>
Quarteto <code>shadow*</code> + <code>elevation</code>	<code>boxShadow</code>
<code>margin: 5</code> em cada filho	<code>gap</code>
<code>Button</code> com <code>color</code> igual nas duas plataformas	iOS tinge o texto , Android o fundo
"Estilize o <code><Button></code> "	<code>Button</code> não aceita <code>style</code>
<code>flexDirection</code> default <code>row</code>	Em RN é <code>column</code>
" <code>create</code> otimiza performance"	Valida tipos e organiza

HANDS-ON – 25 MIN

Exercícios 1 a 6 de `exercises.md`, no projeto do seu grupo.

- ❶ ❶ e ❷ – associação + caça ao erro (sem computador, 8 min)
- ❷ ❸ – `Card` com `boxShadow` e `gap` (7 min)
- ❸ ❹ – previsão de Flexbox: **desenhe antes de rodar** (5 min)
- ❹ ❺ – estilo condicional com `useState`, `Text onPress` e `Switch` (7 min)

NO EXERCÍCIO 4, PREVER ERRADO É O MELHOR MOMENTO DA AULA.

ATIVIDADE APLICADA

Estilizar de verdade a tela do projeto — `exercises.md`, Atividade 1

- `theme.ts` com **tokens**: cores, espaço, tipografia
- `Card` reutilizável com `destacado?` via array de estilos
- Formulário: `TextInput` + `Switch` em `ScrollView`, `Button` que desabilita com o campo vazio
- Um layout Flexbox não trivial, espaçado **só com** `gap`

Proibido: `JSX.Element` · `as const` em `create` · `margin` entre irmãos · quarteto `shadow*` · componente fora dos sete da aula · `any` · cor ou espaço mágico no componente



VOCÊS ENTRARAM COM UMA TELA QUE FUNCIONAVA E UM ESTILO ESPALHADO.

SAEM COM UMA TELA APRESENTÁVEL E UM LUGAR DEFINIDO PARA CADA DECISÃO DE DESIGN.

E SEM CASCATA — PORQUE ELA NUNCA EXISTIU AQUI.



PARA A PRÓXIMA AULA

LEITURA OBRIGATÓRIA

- Cap. 4 — Criando os primeiros componentes
- Cap. 5 — Componentes estilizados
- Cap. 6 — O básico de layout com o Flexbox
- [React Native — Core Components and APIs](#)

RECOMENDADA

- [Flexbox](#) · [StyleSheet](#) · [Layout Props](#)
-  [Flexbox Froggy](#) — ~20 min

Notas completas, com fontes de todos os dados citados: `student-notes.md`

PERGUNTAS?

`student-notes.md` · `exercises.md`